

```

#include <stdio.h>

#include <stdlib.h>

#include <omp.h>

#include <time.h>


#define SIZE 100000


// ----- MERGE FUNCTION
void merge(int arr[], int left, int mid, int right)
{
    int i, j, k;

    int n1 = mid - left + 1;

    int n2 = right - mid;

    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));

    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0; j = 0; k = left;
    while (i < n1 && j < n2) {
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    }
    while (i < n1) {
        arr[k++] = L[i++];
    }
    while (j < n2) {
        arr[k++] = R[j++];
    }
}

```

```

    }

    free(L);

    free(R);
}

// ----- SERIAL MERGE SORT
void serialMergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;

        serialMergeSort(arr, left, mid);

        serialMergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

// ----- PARALLEL MERGE SORT
void parallelMergeSort(int arr[], int left, int right, int depth)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;

        if (depth <= 4)
        {
            #pragma omp parallel sections
            {
                #pragma omp section
                parallelMergeSort(arr, left, mid, depth + 1);
            }
        }
    }
}

```

```

        #pragma omp section
        parallelMergeSort(arr, mid + 1, right, depth + 1);
    }
}
else
{
    // Switch to serial to avoid thread creation overhead
    serialMergeSort(arr, left, mid);
    serialMergeSort(arr, mid + 1, right);
}

merge(arr, left, mid, right);
}
}

// ----- MAIN FUNCTION
int main()
{
    // MUST enable nested parallelism for recursive parallel sections to work
    omp_set_nested(1);

    int *arr_serial = (int *)malloc(SIZE * sizeof(int));
    int *arr_parallel = (int *)malloc(SIZE * sizeof(int));

    // Initialize both arrays with the same random values
    for (int i = 0; i < SIZE; i++)
    {
        int val = rand() % 100000;
        arr_serial[i] = val;
        arr_parallel[i] = val;
    }
}

```

```

}

// ----- SERIAL MERGE SORT -----
// Replaced clock() with omp_get_wtime()
double start_serial = omp_get_wtime();
serialMergeSort(arr_serial, 0, SIZE - 1);
double end_serial = omp_get_wtime();
double time_serial = end_serial - start_serial;

// ----- PARALLEL MERGE SORT -----
// Replaced clock() with omp_get_wtime()
double start_parallel = omp_get_wtime();
parallelMergeSort(arr_parallel, 0, SIZE - 1, 0);
double end_parallel = omp_get_wtime();
double time_parallel = end_parallel - start_parallel;

// ----- OUTPUT -----
printf("Serial Merge Sort Time : %.6f seconds \n", time_serial);
printf("Parallel Merge Sort Time : %.6f seconds \n", time_parallel);

free(arr_serial);
free(arr_parallel);

return 0;
}

```